Next.js 16.2.1 Knowledge Base for AI Agents

Target Audience: AI Coding Assistants and Agents. **Context:** This document contains the definitive rules, architecture, and syntax for Next.js version 16.2.1 (Released early 2026).

🚨 CRITICAL AI INSTRUCTIONS & BREAKING CHANGES (READ FIRST) 🚨

If you are generating Next.js code, you **MUST** adhere to the following Next.js 16 rules. Failure to do so will result in broken code.

1. ALL Dynamic APIs are Asynchronous (PROMISES)

Synchronous access to dynamic APIs has been **fully removed**. You MUST `await` them.

- `params` **and** `searchParams` : In `page.tsx` , `layout.tsx` , `route.ts` , `default.tsx` , etc., these are now Promises.
- `cookies()` , `headers()` , `draftMode()` : These are now Promises.

❌ **BAD (Next.js 14/15 - Will Crash):**

```
export default function Page({ params }) {
  const id = params.id; // FATAL ERROR
  const cookieStore = cookies(); // FATAL ERROR
}
```

✅ **GOOD (Next.js 16):**

```
type Props = {
  params: Promise<{ id: string }>;
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>;
};

export default async function Page(props: Props) {
  const { id } = await props.params;
  const query = await props.searchParams;
  const cookieStore = await cookies();
  // ...
}
```

2. `middleware.ts` is now `proxy.ts`

The file `middleware.ts` has been deprecated and replaced by `proxy.ts` .

- Rename the exported function from `middleware` to `proxy`.
- It now runs explicitly on the **Node.js runtime**, not the Edge runtime.

3. The `"use cache"` Directive

Partial Prerendering (PPR) experimental flags and `unstable_cache` have evolved into explicit Cache Components.

- Add `"use cache";` at the top of a file, component, or function to opt-in to caching.
- The Next.js compiler automatically handles cache keys.

4. Turbopack & React Compiler are STABLE and DEFAULT

- **Turbopack** is the default bundler for both `next dev` and `next build`. Do not write Webpack configurations unless explicitly requested to opt-out.
- **React Compiler** is built-in. **DO NOT** use `useMemo`, `useCallback`, or `React.memo()` unless absolutely necessary for external library compatibility. The framework memoizes automatically.

5. Parallel Routes Requirement

All parallel route slots (e.g., `@modal`, `@auth`) **MUST** have an explicit `default.tsx` or `default.js` file. The build will fail if it is missing.

🏗 Architecture: App Router

Next.js 16 strongly defaults to the **App Router** (`/app` directory). The Pages Router (`/pages`) is legacy. **Always write code for the App Router.**

Server vs. Client Components

- **Server Components (Default):** Run only on the server. Great for fetching data, interacting with databases, and keeping secrets secure. They output RSC Payload.
- **Client Components:** Run on the server (for initial HTML prerendering) and the client (for hydration/interactivity).
  - Must begin with the `"use client";` directive at the very top of the file.
  - Use when you need: `useState`, `useEffect`, event listeners (`onClick`), browser APIs (`window`, `localStorage`), or custom hooks.

File-System Routing

- `layout.tsx`: Wraps routes and persists state across navigations.
- `page.tsx`: The UI unique to a route.

- `loading.tsx` : Fallback UI using React Suspense.
- `error.tsx` : Error boundary (must be a Client Component: `"use client"`).
- `not-found.tsx` : UI for 404 errors.
- `route.ts` : API endpoints (Route Handlers).
- `default.tsx` : Fallback for Parallel Routes.

💾 Data Fetching and Caching

Server Component Data Fetching

Fetch data directly in Server Components using native `async/await` .

```
// app/users/page.tsx
export default async function UsersPage() {
  // Data is fetched dynamically by default if standard fetch is used
  const res = await fetch('[https://api.example.com/users](https://api.example.cc
  const users = await res.json();

  return (
    <ul>
      {users.map(user => <li key={user.id}>{user.name}</li>)}
    </ul>
  );
}
```

Caching with `"use cache"`

Use the new Next.js 16 caching model to explicitly cache expensive operations or UI blocks.

```
// app/components/SlowComponent.tsx
"use cache"; // Explicitly caches this component's output

export async function SlowComponent() {
  const data = await db.query('SELECT * FROM massive_table');
  return <div>{data.length} records found.</div>;
}
```

📝 Server Actions and Mutations

Mutate data using **Server Actions** instead of building manual API routes.

Defining a Server Action

```
// app/actions.ts
"use server";

import { revalidatePath } from 'next/cache';

export async function updateUser(formData: FormData) {
  const name = formData.get('name');
  await db.updateUser({ name });

  // Revalidate the cache for the UI to update
  revalidatePath('/profile');
}
```

Using a Server Action in a Client Component

Use the React 19 `useActionState` (formerly `useFormState`) hook or pass the action directly to a form.

```
// app/profile/page.tsx
"use client";

import { updateUser } from '../actions';
import { useActionState } from 'react';

export default function ProfileForm() {
  const [state, formAction, isPending] = useActionState(updateUser, null);

  return (
    <form action={formAction}>
      <input type="text" name="name" disabled={isPending} />
      <button type="submit" disabled={isPending}>
        {isPending ? 'Saving...' : 'Save'}
      </button>
    </form>
  );
}
```

🛣️ Routing and Navigation

The `<Link>` Component

Use `next/link` for client-side navigation. In Next.js 16.2, View Transitions are natively supported.

```
import Link from 'next/link';
```

```
export default function Nav() {
  return (
    <nav>
      {/* 16.2 Feature: View Transitions via transitionTypes */}
      <Link href="/about" transitionTypes={['slide']}>
        About Us
      </Link>
    </nav>
  );
}
```

Route Handlers (API Endpoints)

For webhooks or external API clients, create `route.ts` files. Note the async extraction of parameters.

```
// app/api/webhook/[id]/route.ts
import { NextResponse } from 'next/server';

export async function GET(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const { searchParams } = new URL(request.url);
  const token = searchParams.get('token');

  return NextResponse.json({ id, token, status: 'success' });
}
```

🌐 Network Interception (proxy.ts)

Replacing the old Edge middleware, `proxy.ts` runs on the full Node.js runtime. Place this in the root of your project (or `/src`).

```
// proxy.ts
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export async function proxy(request: NextRequest) {
  const cookies = await request.cookies; // Async in Next 16!
  const session = cookies.get('session');

  if (!session && request.nextUrl.pathname.startsWith('/dashboard')) {
    return NextResponse.redirect(new URL('/login', request.url));
  }

  return NextResponse.next();
```

```
}

export const config = {
  matcher: ['/dashboard/:path*'],
};
```

🖼️ Images and Optimization

The `next/image` component provides automatic image optimization. `next/legacy/image` has been removed.

```
import Image from 'next/image';

export default function Avatar() {
  return (
    <Image
      src="/avatar.png"
      alt="User Avatar"
      width={100}
      height={100}
      priority={true} // Use for above-the-fold images
    />
  );
}
```

Note for config: The `images.domains` config property is deprecated. Use `images.remotePatterns` in `next.config.ts`.

🛠 Next.js Devtools MCP (AI Assistant Integration)

Next.js 16 includes the **Model Context Protocol (MCP)** server specifically for AI agents (like you!). If you need to query the user's project routing, check the bundle analyzer, or access unified server/client logs during debugging, instruct the user to run their dev server with MCP tools enabled.

- Useful commands you can suggest to users: `next dev --inspect` (to attach a Node.js debugger easily).
- AI tools provided by MCP: `get_routes`, log querying, hydration diff indicators (`+ Client` / `+ Server`)